

# CARLA 0.9.10

## MANUAL TIẾNG VIỆT

Toàn bộ nhóm lệnh chạy, công cụ util, examples và Python API nền tảng

**Phạm vi phiên bản:** Biên soạn riêng cho CARLA 0.9.10 (UE4), ưu tiên Windows package. Các lệnh từ bản mới hơn không được trộn vào tài liệu.

### Ứng dụng trọng tâm

Đồ án học tăng cường sâu điều khiển bám làn trong CARLA

Cập nhật: 14/07/2026 • Ngôn ngữ: Tiếng Việt

## Mục lục tra cứu nhanh

- 1. Phiên bản, cấu trúc và cách dùng manual
- 2. Chuẩn bị Python API trên Windows
- 3. Lệnh khởi chạy CARLA server
- 4. Công cụ PythonAPI/util/config.py
- 5. Danh mục đầy đủ PythonAPI/examples
- 6. Kết nối Client – World – Map – Weather
- 7. Actors, blueprints, spawn và điều khiển xe
- 8. Waypoint, topology và điều hướng
- 9. Sensors và mẫu code lấy dữ liệu
- 10. Synchronous mode, fixed time-step và queue
- 11. Traffic Manager và sinh giao thông
- 12. Recorder, replay và truy vấn sự cố
- 13. Mẫu môi trường bám làn cho đồ án DRL
- 14. Tối ưu cho GTX 1650, 16 GB RAM
- 15. Lỗi thường gặp và checklist vận hành
- Phụ lục A. Bảng API cốt lõi
- Phụ lục B. Nguồn chính thức đúng bản 0.9.10

**Mẹo tra lệnh:** Với mọi script, chạy `python ten_script.py --help` để xem chính xác tham số có trong package của bạn. Đây là nguồn xác nhận cuối cùng nếu package đã được chỉnh sửa.

## 1. Phiên bản, cấu trúc và cách dùng manual

CARLA hoạt động theo kiến trúc server-client. CarlaUE4.exe là server mô phỏng; các file Python trong PythonAPI là client kết nối qua RPC, mặc định cổng 2000 và cổng streaming kế tiếp là 2001.

| Thành phần     | Vai trò                                         | Vị trí thường gặp                        |
|----------------|-------------------------------------------------|------------------------------------------|
| CarlaUE4.exe   | Chạy Unreal Engine và thế giới mô phỏng         | <CARLA_ROOT>\CarlaUE4.exe                |
| Python API egg | Module import carla tương thích 0.9.10          | PythonAPI\carla\dist\carla-0.9.10-...egg |
| examples       | Các ví dụ điều khiển, sensor, traffic, recorder | PythonAPI\examples                       |
| util           | Cấu hình, kiểm tra, benchmark, khám phá lane    | PythonAPI\util                           |
| agents         | BasicAgent, RoamingAgent, BehaviorAgent         | PythonAPI\carla\agents                   |

**Tương thích:** Client và server phải cùng libcarla 0.9.10. Hãy kiểm tra bằng `get_client_version()` và `get_server_version()`; không cài nhầm gói carla mới nhất.

## 2. Chuẩn bị Python API trên Windows

### 2.1 Môi trường khuyến nghị

- Windows 10/11 64-bit.
- Python 3.7 x64 là lựa chọn an toàn nhất cho egg đi kèm package 0.9.10.
- Pygame và NumPy để chạy các examples có giao diện.
- Mở terminal tại đúng thư mục CARLA để đường dẫn tương đối trong examples tìm thấy file egg.

Tạo môi trường Python

```
py -3.7 -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install pygame numpy
```

## 2.2 Kiểm tra file egg

```
cd <CARLA_ROOT>\PythonAPI\carla\dist
dir carla-0.9.10-*.egg
```

Nếu chạy script từ PythonAPI/examples, đoạn glob có sẵn trong examples sẽ tự thêm file egg phù hợp vào sys.path. Nếu viết project ở thư mục khác, đặt PYTHONPATH hoặc thêm đường dẫn trong code.

```
set PYTHONPATH=<CARLA_ROOT>\PythonAPI\carla\dist\carla-0.9.10-py3.7-win-amd64.egg;%PYTHONPATH%
python -c "import carla; print(carla.__file__)"
```

**Không cài nhầm:** Không dùng pip install carla không chỉ định phiên bản, vì pip có thể cài client mới hơn server 0.9.10.

## 3. Lệnh khởi chạy CARLA server

### 3.1 Lệnh cơ bản và cấu hình nhẹ

Mặc định: Epic, RPC 2000

```
cd <CARLA_ROOT>
CarlaUE4.exe
```

Khuyến nghị cho GTX 1650

```
CarlaUE4.exe -quality-level=Low -ResX=800 -ResY=600 -windowed
```

Chế độ demo đẹp

```
CarlaUE4.exe -quality-level=Epic -ResX=1280 -ResY=720 -windowed
```

### 3.2 Các tham số server chính thức của package

| Tham số                 | Ý nghĩa                                       | Ví dụ                   |
|-------------------------|-----------------------------------------------|-------------------------|
| -carla-rpc-port=N       | Cổng client kết nối; streaming mặc định N+1   | -carla-rpc-port=3000    |
| -carla-streaming-port=N | Cổng truyền dữ liệu sensor; 0 chọn cổng trống | -carla-streaming-port=0 |
| -quality-level=Low      | Tắt hậu kỳ/bóng, giảm khoảng vẽ để tăng FPS   | -quality-level=Low      |
| -quality-level=Epic     | Đồ họa đầy đủ, mặc định                       | -quality-level=Epic     |
| -ResX / -ResY           | Kích thước cửa sổ UE4                         | -ResX=800 -ResY=600     |
| -windowed               | Chạy cửa sổ thay vì toàn màn hình             | -windowed               |

### 3.3 Chạy nhiều CARLA server

```
:: Server A
CarlaUE4.exe -carla-rpc-port=4000 -quality-level=Low
```

```
:: Server B
CarlaUE4.exe -carla-rpc-port=5000 -quality-level=Low
```

**Cổng:** Mỗi server cần cặp cổng riêng. Server dùng 4000 thường dùng streaming 4001; Traffic Manager phải dùng cổng khác, ví dụ 4050.

### 3.4 No-rendering và off-screen

No-rendering là world setting, không phải chỉ ẩn cửa sổ. Khi bật, GPU sensors như camera trả dữ liệu rỗng. Nó phù hợp nếu agent RL lấy state từ waypoint, transform, velocity và collision/lane events.

```
cd <CARLA_ROOT>\PythonAPI\util
python config.py --no-rendering
python config.py --rendering
```

**CARLA 0.9.10 trên Windows:** Tài liệu 0.9.10 chỉ mô tả off-screen chính thức cho Linux + OpenGL. Trên Windows, hãy dùng cửa sổ nhỏ/Low hoặc bật no-rendering qua world settings.

## 4. Công cụ PythonAPI/util/config.py

config.py là công cụ dòng lệnh quan trọng nhất để xem và đổi world đang chạy. Các tham số dưới đây được trích từ source 0.9.10.

| Tham số                      | Chức năng                                                              |
|------------------------------|------------------------------------------------------------------------|
| --host H                     | IP server, mặc định localhost                                          |
| -p, --port P                 | RPC port, mặc định 2000                                                |
| -d, --default                | Trả world về rendering bật, variable time-step, weather Default, async |
| -m, --map MAP                | Nạp map mới                                                            |
| -r, --reload-map             | Nạp lại map hiện tại                                                   |
| --delta-seconds S            | Fixed delta; 0 chuyển variable time-step                               |
| --fps N                      | Đặt fixed FPS; 0 chuyển variable FPS                                   |
| --rendering / --no-rendering | Bật/tắt rendering                                                      |
| --no-sync                    | Tắt synchronous mode                                                   |
| --weather PRESET             | Đặt weather preset                                                     |
| -i, --inspect                | In map, weather, frame rate, mode và số actor                          |
| -l, --list                   | Liệt kê maps và weather presets                                        |
| -b, --list-blueprints FILTER | Liệt kê blueprint theo wildcard                                        |
| -x, --xodr-path FILE         | Tạo world từ OpenDRIVE                                                 |
| --osm-path FILE              | Chuyển OSM sang OpenDRIVE và tạo world                                 |

Nhóm lệnh dùng thường xuyên

```
cd <CARLA_ROOT>\PythonAPI\util

python config.py --help
python config.py --list
python config.py --inspect
python config.py --map Town01
python config.py --reload-map
python config.py --weather ClearNoon
python config.py --fps 20
python config.py --delta-seconds 0.05
python config.py --list-blueprints "vehicle.*"
python config.py --list-blueprints "sensor.*"
python config.py --default
```

**Synchronous mode:** config.py 0.9.10 chỉ có --no-sync để tắt. Khi cần bật sync, client điều khiển chính phải đặt settings.synchronous\_mode=True và tự gọi world.tick().

## 5. Danh mục đầy đủ PythonAPI/examples

Danh sách dưới đây bám theo tree tag 0.9.10. Tên sensor\_synchronization.py trong repository được viết thiếu chữ h; hãy dùng đúng tên file có trong package.

| File                            | Mục đích                                        | Lệnh mẫu                                                              |
|---------------------------------|-------------------------------------------------|-----------------------------------------------------------------------|
| automatic_control.py            | Chạy Basic/Roaming/Behavior agent tới đích      | python automatic_control.py --agent Behavior --behavior normal --loop |
| client_bounding_boxes.py        | Chiều bounding box 3D của xe lên ảnh camera     | python client_bounding_boxes.py                                       |
| dynamic_weather.py              | Thay đổi thời tiết và mặt trời liên tục         | python dynamic_weather.py --speed 1.0                                 |
| manual_control.py               | Điều khiển xe bằng WASD, xem nhiều sensor       | python manual_control.py --res 800x600                                |
| manual_control_steeringwheel.py | Điều khiển bằng vô-lăng/joystick                | python manual_control_steeringwheel.py                                |
| no_rendering_mode.py            | Hiển thị bản đồ 2D bằng Pygame khi no-rendering | python no_rendering_mode.py                                           |
| open3d_lidar.py                 | Hiển thị LiDAR bằng Open3D                      | python open3d_lidar.py                                                |
| sensor_synchronization.py       | Đồng bộ dữ liệu nhiều sensor cùng frame         | python sensor_synchronization.py                                      |
| show_recorder_actors_blocked.py | Tìm actor bị kẹt trong recorder                 | python show_recorder_actors_blocked.py -f run.log -t 60 -d 100        |
| show_recorder_collisions.py     | Lọc va chạm trong recorder                      | python show_recorder_collisions.py -f run.log -t va                   |
| show_recorder_file_info.py      | Xem nội dung file recorder                      | python show_recorder_file_info.py -f run.log -s                       |
| spawn_npc.py                    | Sinh xe, walker và Traffic Manager              | python spawn_npc.py -n 10 -w 0 --safe                                 |
| start_recording.py              | Bắt đầu ghi mô phỏng                            | python start_recording.py -f run.log -n 10 -t 60                      |
| start_replaying.py              | Phát lại file recorder                          | python start_replaying.py -f run.log -s 0 -d 0 -c 0                   |
| synchronous_mode.py             | Ví dụ world + camera chạy synchronous           | python synchronous_mode.py                                            |

| File               | Mục đích                                    | Lệnh mẫu                                |
|--------------------|---------------------------------------------|-----------------------------------------|
| tutorial.py        | Ví dụ nhập môn: spawn xe, camera, autopilot | python tutorial.py                      |
| vehicle_gallery.py | Duyệt và trưng bày các mẫu xe               | python vehicle_gallery.py               |
| vehicle_physics.py | Đọc/chỉnh VehiclePhysicsControl             | python vehicle_physics.py               |
| rss\...            | Ví dụ RSS nếu dùng package/build có RSS     | python rss\manual_control_rss.py --help |

## 5.1 Tham số quan trọng của spawn\_npc.py

| Tham số                    | Mặc định / ý nghĩa                        |
|----------------------------|-------------------------------------------|
| -n, --number-of-vehicles N | 10 xe                                     |
| -w, --number-of-walkers W  | 50 người đi bộ                            |
| --safe                     | Loại xe dễ gây tai nạn/không phù hợp      |
| --filterv PATTERN          | vehicle.*                                 |
| --filterw PATTERN          | walker.pedestrian.*                       |
| --tm-port P                | 8000                                      |
| --sync                     | Traffic Manager và world chạy synchronous |
| --hybrid                   | Bật hybrid physics                        |
| -s, --seed S               | Seed tái lập                              |
| --car-lights-on            | Bật đèn xe                                |

```
cd <CARLA_ROOT>\PythonAPI\examples
python spawn_npc.py -n 5 -w 0 --safe --hybrid --seed 42
python spawn_npc.py -n 15 -w 10 --safe --car-lights-on
python spawn_npc.py --port 4000 --tm-port 4050 -n 10 -w 0
```

## 5.2 automatic\_control.py

```
python automatic_control.py --agent Behavior --behavior cautious --loop
python automatic_control.py --agent Behavior --behavior normal --loop --seed 42
python automatic_control.py --agent Basic --res 800x600
python automatic_control.py --agent Roaming --filter "vehicle.tesla.*"
```

Các lựa chọn agent: Behavior, Roaming, Basic. Behavior có ba phong cách cautious, normal, aggressive.

## 5.3 manual\_control.py: phím điều khiển

| Phím  | Chức năng            |
|-------|----------------------|
| W / S | Ga / phanh           |
| A / D | Đánh lái trái / phải |
| Q     | Tiến/lùi             |
| Space | Phanh tay            |

| Phím        | Chức năng                |
|-------------|--------------------------|
| P           | Bật/tắt autopilot        |
| M           | Số tay                   |
| , / .       | Giảm / tăng số           |
| Tab         | Đổi vị trí camera        |
| ` hoặc N    | Sensor kế tiếp           |
| 1-9         | Chọn sensor              |
| G           | Radar                    |
| C / Shift+C | Đổi thời tiết            |
| Backspace   | Đổi xe                   |
| R           | Ghi ảnh                  |
| Ctrl+R      | Ghi simulation           |
| Ctrl+P      | Replay recorder gần nhất |
| F1          | HUD                      |
| H hoặc ?    | Help                     |
| Esc         | Thoát                    |

## 6. Kết nối Client - World - Map - Weather

### 6.1 Kết nối và kiểm tra phiên bản

```
import carla

client = carla.Client('127.0.0.1', 2000)
client.set_timeout(10.0)
world = client.get_world()

print('Client:', client.get_client_version())
print('Server:', client.get_server_version())
print('Map:', world.get_map().name)
```

### 6.2 Liệt kê và nạp map

```
print(client.get_available_maps())
world = client.load_world('Town01')
# Nạp lại map hiện tại và xóa actor động:
world = client.reload_world()
```

**load\_world:** Nạp map mới sẽ hủy world/episode cũ. Mọi actor reference cũ không còn hợp lệ.

## 6.3 Weather

```
world.set_weather(carla.WeatherParameters.ClearNoon)
world.set_weather(carla.WeatherParameters.WetCloudySunset)

custom = carla.WeatherParameters(
    cloudiness=30.0,
    precipitation=0.0,
    sun_altitude_angle=45.0)
world.set_weather(custom)
```

## 6.4 Spectator camera

```
spectator = world.get_spectator()
spectator.set_transform(carla.Transform(
    carla.Location(x=0.0, y=0.0, z=50.0),
    carla.Rotation(pitch=-90.0)))
```

# 7. Actors, blueprints, spawn và điều khiển xe

## 7.1 Liệt kê blueprint

```
library = world.get_blueprint_library()
for bp in library.filter('vehicle.*'):
    print(bp.id)

tesla_bp = library.find('vehicle.tesla.model3')
if tesla_bp.has_attribute('color'):
    tesla_bp.set_attribute('color', '0,0,255')
tesla_bp.set_attribute('role_name', 'hero')
```

## 7.2 Spawn và destroy đúng cách

```
spawn_points = world.get_map().get_spawn_points()
vehicle = world.try_spawn_actor(tesla_bp, spawn_points[0])

if vehicle is None:
    raise RuntimeError('Spawn point đang bị chiếm')

try:
    print(vehicle.id, vehicle.get_transform())
finally:
    vehicle.destroy()
```

## 7.3 Điều khiển trực tiếp

```
control = carla.VehicleControl(
    throttle=0.35,
    steer=0.0,
    brake=0.0,
    hand_brake=False,
    reverse=False)
vehicle.apply_control(control)

# Autopilot qua Traffic Manager mặc định port 8000
vehicle.set_autopilot(True, 8000)
```



| Trường VehicleControl | Khoảng / ý nghĩa        |
|-----------------------|-------------------------|
| throttle              | 0.0–1.0                 |
| steer                 | -1.0 trái đến +1.0 phải |
| brake                 | 0.0–1.0                 |
| hand_brake            | bool                    |
| reverse               | bool                    |
| manual_gear_shift     | bool                    |
| gear                  | số nguyên               |

## 7.4 Batch commands

```
batch = [carla.command.DestroyActor(actor.id) for actor in actors]
client.apply_batch(batch)

# apply_batch_sync có thể yêu cầu tick sau batch
responses = client.apply_batch_sync(batch, True)
for response in responses:
    if response.error:
        print(response.error)
```

## 8. Waypoint, topology và điều hướng

### 8.1 Waypoint gần xe

```
carla_map = world.get_map()
location = vehicle.get_location()
wp = carla_map.get_waypoint(
    location,
    project_to_road=True,
    lane_type=carla.LaneType.Driving)

print(wp.road_id, wp.section_id, wp.lane_id)
print(wp.lane_width, wp.is_junction)
next_wps = wp.next(2.0)
```

### 8.2 Lane bên cạnh và topology

```
left_wp = wp.get_left_lane()
right_wp = wp.get_right_lane()

topology = carla_map.get_topology()
for entry_wp, exit_wp in topology:
    print(entry_wp.road_id, '->', exit_wp.road_id)
```

## 8.3 Vẽ waypoint để debug

```
world.debug.draw_point(
    wp.transform.location,
    size=0.12,
    color=carla.Color(0, 255, 0),
    life_time=0.2)

world.debug.draw_arrow(
    wp.transform.location,
    wp.transform.location + wp.transform.get_forward_vector() * 2.0,
    thickness=0.05,
    arrow_size=0.1,
    color=carla.Color(255, 0, 0),
    life_time=0.2)
```

## 9. Sensors và mẫu code lấy dữ liệu

### 9.1 Danh mục sensor quan trọng trong 0.9.10

| Blueprint                           | Output                         | Ứng dụng                        |
|-------------------------------------|--------------------------------|---------------------------------|
| sensor.camera.rgb                   | carla.Image                    | Ảnh màu                         |
| sensor.camera.depth                 | carla.Image                    | Khoảng cách theo pixel          |
| sensor.camera.semantic_segmentation | carla.Image                    | Nhãn lớp pixel                  |
| sensor.camera.dvs                   | carla.DVSEventArray            | Event camera                    |
| sensor.lidar.ray_cast               | carla.LidarMeasurement         | Point cloud + intensity         |
| sensor.lidar.ray_cast_semantic      | carla.SemanticLidarMeasurement | Point cloud có nhãn             |
| sensor.other.collison               | carla.CollisionEvent           | Phạt va chạm / kết thúc episode |
| sensor.other.lane_invasion          | carla.LaneInvasionEvent        | Phát hiện vượt vạch             |
| sensor.other.gnss                   | carla.GnssMeasurement          | Tọa độ địa lý                   |
| sensor.other.imu                    | carla.IMUMeasurement           | Gia tốc, gyro, compass          |
| sensor.other.obstacle               | carla.ObstacleDetectionEvent   | Vật cản phía trước              |
| sensor.other.radar                  | carla.RadarMeasurement         | Vận tốc tương đối, góc, depth   |

## 9.2 Camera RGB và lưu ảnh

```
camera_bp = world.get_blueprint_library().find('sensor.camera.rgb')
camera_bp.set_attribute('image_size_x', '640')
camera_bp.set_attribute('image_size_y', '360')
camera_bp.set_attribute('fov', '90')
camera_bp.set_attribute('sensor_tick', '0.05')

camera_tf = carla.Transform(
    carla.Location(x=1.5, z=2.2),
    carla.Rotation(pitch=-8.0))

camera = world.spawn_actor(camera_bp, camera_tf, attach_to=vehicle)
camera.listen(lambda image: image.save_to_disk(
    '_out/%06d.png' % image.frame))
```

## 9.3 Chuyển ảnh CARLA sang NumPy

```
import numpy as np

def image_to_bgr(image):
    array = np.frombuffer(image.raw_data, dtype=np.uint8)
    array = array.reshape((image.height, image.width, 4))
    return array[:, :, :3] # CARLA raw là BGRA

camera.listen(lambda image: process(image_to_bgr(image)))
```

## 9.4 Collision và lane invasion

```
collision_bp = world.get_blueprint_library().find('sensor.other.collision')
collision = world.spawn_actor(collision_bp, carla.Transform(), attach_to=vehicle)

collision_flag = {'hit': False}
collision.listen(lambda event: collision_flag.update(hit=True))

lane_bp = world.get_blueprint_library().find('sensor.other.lane_invasion')
lane_sensor = world.spawn_actor(lane_bp, carla.Transform(), attach_to=vehicle)
lane_sensor.listen(lambda event: print(event.crossed_lane_markings))
```

## 9.5 Dọn sensor

```
for sensor in sensors:
    if sensor.is_listening:
        sensor.stop()
        sensor.destroy()
vehicle.destroy()
```

**Quy tắc cleanup:** Luôn stop sensor trước destroy, và đặt cleanup trong finally. Nếu không, callback/Python reference có thể giữ tài nguyên sau mỗi episode.

## 10. Synchronous mode, fixed time-step và queue

Đây là chế độ nên dùng cho thu thập dữ liệu và RL: client làm chủ từng bước; world và sensors cùng gắn với frame cụ thể.

## 10.1 Mẫu loop chuẩn

```
import queue

settings = world.get_settings()
settings.synchronous_mode = True
settings.fixed_delta_seconds = 0.05 # 20 FPS mô phỏng
world.apply_settings(settings)

image_queue = queue.Queue()
camera.listen(image_queue.put)

try:
    while True:
        frame = world.tick()
        image = image_queue.get(timeout=2.0)
        if image.frame != frame:
            print('Frame lệch:', frame, image.frame)
finally:
    camera.stop()
    camera.destroy()
    settings = world.get_settings()
    settings.synchronous_mode = False
    settings.fixed_delta_seconds = None
    world.apply_settings(settings)
```

**Giới hạn vật lý:** Tài liệu 0.9.10 cảnh báo không dùng `fixed_delta_seconds > 0.1` s. Với đề án, 0.05 s (20 Hz) là lựa chọn cân bằng.

## 10.2 Đồng bộ nhiều sensor

```
import queue

q = queue.Queue()
world.on_tick(q.put)
camera.listen(q.put)
imu.listen(q.put)

def retrieve(q, frame, timeout=2.0):
    while True:
        data = q.get(timeout=timeout)
        if data.frame == frame:
            return data

frame = world.tick()
snapshot = retrieve(q, frame)
image = retrieve(q, frame)
imu_data = retrieve(q, frame)
```

**Camera delay:** GPU sensors thường trễ vài frame. Trong dự án thật nên dùng queue riêng theo sensor và lọc theo frame, tránh giả định callback đến đúng thứ tự.

## 11. Traffic Manager và sinh giao thông

### 11.1 Tạo Traffic Manager

```
tm = client.get_trafficmanager(8000)
tm.set_global_distance_to_leading_vehicle(3.0)
tm.global_percentage_speed_difference(20.0)

vehicle.set_autopilot(True, tm.get_port())
```

### 11.2 Chinh hành vi theo xe

```
tm.distance_to_leading_vehicle(vehicle, 5.0)
tm.vehicle_percentage_speed_difference(vehicle, 10.0)
tm.auto_lane_change(vehicle, False)
tm.ignore_lights_percentage(vehicle, 0.0)
tm.ignore_walkers_percentage(vehicle, 0.0)
tm.force_lane_change(vehicle, True) # True: phải, False: trái
```

### 11.3 Sync và deterministic

```
tm.set_synchronous_mode(True)
tm.set_random_device_seed(42)

settings = world.get_settings()
settings.synchronous_mode = True
settings.fixed_delta_seconds = 0.05
world.apply_settings(settings)

while training:
    world.tick()
```

**Chỉ một tick master:** Trong multi-client, chỉ một client được gọi world.tick(). Nếu nhiều client cùng tick, server coi tất cả tick như nhau và gây sai đồng bộ.

### 11.4 Hybrid physics

```
tm.set_hybrid_physics_mode(True)
tm.set_hybrid_mode_radius(70.0)
```

Hybrid physics chỉ bật vật lý trong vùng gần xe role\_name="hero"; xe xa di chuyển bằng cập nhật vị trí. Đây là lựa chọn tốt khi demo nhiều NPC trên laptop.

## 12. Recorder, replay và truy vấn sự cố

### 12.1 API recorder

```
client.start_recorder('thesis_run.log', True)
# ... chạy mô phỏng ...
client.stop_recorder()

# start=0, duration=0 (toàn bộ), camera actor id=0
client.replay_file('thesis_run.log', 0, 0, 0)
client.set_replayer_time_factor(1.0)
# CARLA 0.9.10 có client.stop_replayer()
client.stop_replayer()
```

## 12.2 Các script recorder

```
cd <CARLA_ROOT>\PythonAPI\examples

python start_recording.py -f thesis_run.log -n 10 -t 60
python start_replaying.py -f thesis_run.log -s 0 -d 0 -c 0
python show_recorder_file_info.py -f thesis_run.log -s
python show_recorder_collisions.py -f thesis_run.log -t va
python show_recorder_actors_blocked.py -f thesis_run.log -t 60 -d 100
```

| Mã lọc | Actor         |
|--------|---------------|
| h      | Hero          |
| v      | Vehicle       |
| w      | Walker        |
| t      | Traffic light |
| o      | Other         |
| a      | Any           |

**Vị trí file:** Nếu filename không chứa dấu /, \ hoặc :, recorder được lưu phía server trong CarlaUE4/Saved.

## 13. Mẫu môi trường bấm làn cho đồ án DRL

### 13.1 State đề xuất

| Biến           | Cách lấy                                                | Chuẩn hóa gợi ý   |
|----------------|---------------------------------------------------------|-------------------|
| lateral_error  | Khoảng cách có dấu từ xe tới tâm waypoint               | chia 3.5 m        |
| heading_error  | Góc yaw xe - yaw waypoint                               | chia 180° hoặc pi |
| speed          | Độ lớn vehicle.get_velocity()                           | chia 15-20 m/s    |
| steer_prev     | Action steer bước trước                                 | đã trong [-1,1]   |
| curvature_hint | Chênh yaw giữa waypoint hiện tại và waypoint phía trước | chia 45°          |

## 13.2 Tính sai lệch tâm làn và góc hướng

```
import math
import numpy as np

def wrap_deg(angle):
    return (angle + 180.0) % 360.0 - 180.0

def lane_state(vehicle, carla_map):
    tf = vehicle.get_transform()
    wp = carla_map.get_waypoint(
        tf.location,
        project_to_road=True,
        lane_type=carla.LaneType.Driving)

    lane_loc = wp.transform.location
    right = wp.transform.get_right_vector()
    offset = tf.location - lane_loc
    lateral_error = offset.x * right.x + offset.y * right.y
    heading_error = wrap_deg(tf.rotation.yaw - wp.transform.rotation.yaw)

    v = vehicle.get_velocity()
    speed = math.sqrt(v.x*v.x + v.y*v.y + v.z*v.z)
    return np.array([lateral_error, heading_error, speed], dtype=np.float32)
```

## 13.3 Action và reward mẫu

```
def apply_action(vehicle, action):
    steer = float(np.clip(action[0], -1.0, 1.0))
    throttle = float(np.clip(action[1], 0.0, 1.0))
    brake = float(np.clip(action[2], 0.0, 1.0))
    vehicle.apply_control(carla.VehicleControl(
        steer=steer, throttle=throttle, brake=brake))

def reward_fn(lateral_error, heading_error_deg, speed,
    steer, steer_prev, collision):
    reward = 1.0
    reward -= 0.55 * abs(lateral_error)
    reward -= 0.015 * abs(heading_error_deg)
    reward -= 0.08 * abs(steer - steer_prev)
    reward -= 0.10 * abs(speed - 8.0)
    if collision:
        reward -= 100.0
    return reward
```

## 13.4 Khung step/reset

```
class CarlaLaneEnv:
    def reset(self):
        self.destroy_actors()
        self.spawn_ego_and_sensors()
        self.collision = False
        self.steer_prev = 0.0
        for _ in range(10):
            self.world.tick()
        return self.get_observation()

    def step(self, action):
        apply_action(self.vehicle, action)
        frame = self.world.tick()
        obs = self.get_observation()
        reward = self.compute_reward(action)
        done = (self.collision or
                abs(obs[0]) > 1.75 or
                self.episode_steps >= self.max_steps)
        info = {'frame': frame, 'collision': self.collision}
        self.episode_steps += 1
        return obs, reward, done, info
```

**Thiết kế khuyến nghị:** Huấn luyện bằng state vector + no-rendering; bật RGB camera và rendering khi đánh giá/demo. Cách này phù hợp GTX 1650 và vẫn là Deep RL nếu policy/value network là mạng sâu.

## 14. Tối ưu cho GTX 1650, 16 GB RAM

| Hạng mục          | Huấn luyện                        | Demo                     |
|-------------------|-----------------------------------|--------------------------|
| Quality           | Low                               | Low hoặc Epic nếu đủ FPS |
| Resolution server | 800×600 windowed                  | 1280×720                 |
| RGB sensor        | Tắt hoặc 320×180                  | 640×360                  |
| World FPS         | 20 Hz, delta 0.05                 | 20–30 Hz                 |
| NPC vehicles      | 0–5                               | 5–15 + hybrid            |
| Walkers           | 0                                 | 0–10                     |
| State             | Waypoint/vector                   | Vector + camera hiển thị |
| Rendering         | No-rendering nếu không cần camera | Bật                      |

### Preset thực tế cho máy của bạn

```
:: Terminal 1 - server nhẹ
CarlaUE4.exe -quality-level=Low -ResX=800 -ResY=600 -windowed

:: Terminal 2 - kiểm tra và map nhẹ
cd PythonAPI\util
python config.py --map Town01
python config.py --inspect

:: Terminal 3 - traffic tối thiểu cho demo
cd ../examples
python spawn_npc.py -n 5 -w 0 --safe --hybrid --seed 42
```



## 15. Lỗi thường gặp và checklist vận hành

| Triệu chứng                         | Nguyên nhân thường gặp                          | Cách xử lý                                                |
|-------------------------------------|-------------------------------------------------|-----------------------------------------------------------|
| ModuleNotFoundError: carla          | Python không thấy egg hoặc sai phiên bản Python | Dùng Python 3.7 x64; chạy từ examples hoặc đặt PYTHONPATH |
| Timeout while waiting for simulator | Server chưa mở, sai port, load map lâu          | Mở CarlaUE4.exe; kiểm tra port; tăng timeout 10-30 s      |
| Client/Server version mismatch      | Cài pip carla khác 0.9.10                       | Gỡ gói sai; dùng egg đi kèm package                       |
| Sensor không có dữ liệu             | No-rendering hoặc sensor chưa listen            | Bật rendering cho camera; kiểm tra callback               |
| World đứng im                       | Sync mode nhưng không có tick master            | Client chính phải gọi world.tick()                        |
| FPS tụt dần                         | Nhiệt/VRAM/RAM, quá nhiều actor/sensor          | Low, giảm resolution/NPC, hybrid, dọn actor               |
| Actor remains after script          | Thiếu finally/destroy                           | Theo dõi actor list và cleanup batch                      |
| SpawnActor failed                   | Spawn point bị chiếm                            | Dùng try_spawn_actor và đổi spawn point                   |
| Ảnh lệch frame                      | Camera GPU có độ trễ                            | Sync + queue theo frame                                   |

### Checklist mỗi lần chạy

1. Cắm sạc, bật chế độ hiệu năng cao và quạt mạnh.
2. Mở CarlaUE4.exe bằng GTX 1650, chế độ Low 800×600 khi phát triển.
3. Chạy config.py --inspect và xác nhận server version 0.9.10.
4. Nếu huấn luyện: bật synchronous + fixed\_delta\_seconds=0.05 trong client chính.
5. Nếu dùng Traffic Manager sync: bật sync cho cả TM và world.
6. Chỉ một client được gọi world.tick().
7. Đưa sensor data qua queue; kiểm tra frame.
8. Trong finally: stop sensor, destroy sensors/vehicles, trả world về async.
9. Theo dõi RAM, Dedicated GPU Memory và nhiệt độ GPU.

## Phụ lục A. Bảng API cốt lõi

| Class/namespace | Nhóm method cần nhớ                                                                                                                                    |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| carla.Client    | get_world, load_world, reload_world, get_available_maps, get_trafficmanager, apply_batch, apply_batch_sync, start_recorder, replay_file                |
| carla.World     | get_settings, apply_settings, tick, wait_for_tick, get_snapshot, get_map, get_actors, get_blueprint_library, spawn_actor, try_spawn_actor, set_weather |
| carla.Actor     | get_transform, set_transform, get_location, get_velocity, destroy, set_simulate_physics                                                                |
| carla.Vehicle   | apply_control, get_control, set_autopilot, get_speed_limit, get_physics_control, apply_physics_control                                                 |
| carla.Sensor    | listen, stop, is_listening, destroy                                                                                                                    |
| carla.Map       | get_spawn_points, get_waypoint, get_topology, generate_waypoints, to_opendrive                                                                         |

| Class/namespace      | Nhóm method cần nhớ                                                                                                                               |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| carla.Waypoint       | next, previous, get_left_lane, get_right_lane, transform, lane_id, road_id, lane_width, is_junction                                               |
| carla.TrafficManager | set_synchronous_mode, set_random_device_seed, global_percentage_speed_difference, set_global_distance_to_leading_vehicle, set_hybrid_physics_mode |
| carla.DebugHelper    | draw_point, draw_line, draw_arrow, draw_box, draw_string                                                                                          |
| carla.command        | SpawnActor, DestroyActor, ApplyVehicleControl, SetAutopilot, SetVehicleLightState, FutureActor                                                    |

**API đầy đủ:** Python API 0.9.10 có hàng nghìn dòng reference. Tra cứu chữ ký, type và return value tại liên kết chính thức trong Phụ lục B.

## Phụ lục B. Nguồn chính thức đúng bản 0.9.10

- Trang chủ tài liệu 0.9.10: <https://carla.readthedocs.io/en/0.9.10/>
- Quick start và command-line options: [https://carla.readthedocs.io/en/0.9.10/start\\_quickstart/](https://carla.readthedocs.io/en/0.9.10/start_quickstart/)
- Python API reference: [https://carla.readthedocs.io/en/0.9.10/python\\_api/](https://carla.readthedocs.io/en/0.9.10/python_api/)
- Sensors reference: [https://carla.readthedocs.io/en/0.9.10/ref\\_sensors/](https://carla.readthedocs.io/en/0.9.10/ref_sensors/)
- Blueprint Library: [https://carla.readthedocs.io/en/0.9.10/bp\\_library/](https://carla.readthedocs.io/en/0.9.10/bp_library/)
- Code recipes: [https://carla.readthedocs.io/en/0.9.10/ref\\_code\\_recipes/](https://carla.readthedocs.io/en/0.9.10/ref_code_recipes/)
- Synchrony và time-step: [https://carla.readthedocs.io/en/0.9.10/adv\\_synchrony\\_timestep/](https://carla.readthedocs.io/en/0.9.10/adv_synchrony_timestep/)
- Rendering options: [https://carla.readthedocs.io/en/0.9.10/adv\\_rendering\\_options/](https://carla.readthedocs.io/en/0.9.10/adv_rendering_options/)
- Traffic Manager: [https://carla.readthedocs.io/en/0.9.10/adv\\_traffic\\_manager/](https://carla.readthedocs.io/en/0.9.10/adv_traffic_manager/)
- Recorder: [https://carla.readthedocs.io/en/0.9.10/adv\\_recorder/](https://carla.readthedocs.io/en/0.9.10/adv_recorder/)
- GitHub examples đúng tag 0.9.10: <https://github.com/carla-simulator/carla/tree/0.9.10/PythonAPI/examples>
- GitHub util đúng tag 0.9.10: <https://github.com/carla-simulator/carla/tree/0.9.10/PythonAPI/util>
- Release CARLA 0.9.10: <https://github.com/carla-simulator/carla/releases/tag/0.9.10>